SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: Database Management Systems
COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO3: *Analyze relational databases using functional dependencies, normalization (1NF to 5NF), and key constraints to identify redundancy and ensure data integrity.* (BL2, BL3)

Activity Tool : Case Study (Medium)
Assessment Tool Weightage: 30%

Dr.G.Ayyappan

QUESTIONS			Total Marks: 50
Q.No	**Question**	**Bloom's Level**	**Marks**
1	Analyze the case: An online shopping platform stores Order(OrderID, CustID, CustName, ProdID, ProdName, Qty, Price). Identify FDs and normalize to 3NF.	BL2 – Understand	**5**
2	A hospital maintains Patient(PID, PName, DocID, DocName, Specialization, WardNo, WardName). Identify anomalies and normalize to BCNF.	BL3 – Apply	**5**
3	Case study: A university stores Enrollment(SID, SName, CourseID, CourseName, Faculty, Grade). Analyze FDs and apply 2NF decomposition.	BL3 – Apply	**5**
4	Given the airline booking case: Booking(FlightNo, PassID, PassName, Seat, DeptCity, ArrCity, Date). Normalize up to BCNF.	BL3 – Apply	**5**
5	Analyze an HR system schema for a multinational company. Identify all functional and transitive dependencies and normalize to 3NF.	BL2 – Understand	**5**
6	Case: Library(MemberID, MemberName, BookID, Title, Author, Genre, BorrowDate). Identify MVDs and decompose to 4NF.	BL3 – Apply	**5**

7	Analyze a telecom billing schema and identify all candidate keys, primary keys, and functional dependencies before normalization.	BL2 – Understand	**5**
8	A retail inventory system has Product(PID, PName, SupplierID, SupplierName, Category, Price, Warehouse). Apply complete normalization.	BL3 – Apply	**5**
9	Study the given poorly normalized schema for a School ERP and propose a fully normalized design up to BCNF with justification.	BL3 – Apply	**5**
10	Given a movie streaming platform schema, identify join dependencies and decompose to 5NF with lossless-join verification.	BL3 – Apply	**5**

Code of Conduct:

I E . B . A S H W I N I / 1 9 2 5 1 1 4 2 2 ____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:
Ashwini

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning

Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand

1.Analyze the case: An online shopping platform stores Order(OrderID, CustID, CustName, ProdID, ProdName, Qty, Price). Identify FDs and normalize to 3NF.

Given Relation

ORDER(OrderID, CustID, CustName, ProdID, ProdName, Qty, Price)

The online shopping platform stores customer, order, and product information in a single relation. This causes redundancy because customer and product details may be repeated for every order.

Step 1: Identify Functional Dependencies

Assume the following business rules:

- Each OrderID identifies one customer.
- Each CustID identifies exactly one customer.
- Each ProdID identifies one product.
- A product has one product name and price.
- An order can contain multiple products.
- Quantity depends on the particular order and product.

Therefore:

1. OrderID → CustID
2. CustID → CustName
3. ProdID → ProdName
4. ProdID → Price
5. (OrderID, ProdID) → Qty

Step 2: Find Candidate Key

An order can contain several products, so OrderID alone cannot uniquely identify a row.

Similarly, ProdID alone cannot identify an order.

Therefore:

Candidate Key = (OrderID, ProdID)

All attributes can be determined by this composite key.

Step 3: Analyze 1NF

The relation is in **1NF** because:

- Each field contains a single atomic value.
- There are no repeating groups.
- Each row represents one product in an order.

Step 4: Analyze 2NF

For 2NF, no non-key attribute should depend on only part of the composite key.

But:

OrderID → CustID

and

ProdID → ProdName, Price

These are partial dependencies.

Therefore, the relation is not in 2NF.

Step 5: Decompose into 2NF

Create:

ORDER

Order(OrderID, CustID)

PRODUCT

Product(ProdID, ProdName, Price)

ORDER_ITEM

Order_Item(OrderID, ProdID, Qty)

The customer dependency still exists:

CustID → CustName

Step 6: Analyze 3NF

There is a transitive dependency:

OrderID → CustID

and

CustID → CustName

Therefore:

OrderID → CustName

This means CustName depends transitively on OrderID.

Step 7: Final 3NF Decomposition

CUSTOMER

Customer(CustID, CustName)

Primary Key: CustID

ORDER

Order(OrderID, CustID)

Primary Key: OrderID

Foreign Key: CustID

PRODUCT

Product(ProdID, ProdName, Price)

Primary Key: ProdID

ORDER_ITEM

Order_Item(OrderID, ProdID, Qty)

Primary Key: (OrderID, ProdID)

Anomalies in Original Relation

Updateanomaly:

If a customer's name changes, every order belonging to that customer must be updated.

Insertionanomaly:

A new customer cannot be stored until the customer places an order.

Deletionanomaly:

If all orders of a customer are deleted, the customer's information may also be lost.

Final Answer

The normalized schema is:

CUSTOMER(CustID, CustName)

ORDER(OrderID, CustID)

PRODUCT(ProdID, ProdName, Price)

ORDER_ITEM(OrderID, ProdID, Qty)

The decomposition removes partial and transitive dependencies and reduces redundancy. Therefore, the design satisfies 3NF.

2. A hospital maintains Patient(PID, PName, DocID, DocName, Specialization, WardNo, WardName). Identify anomalies and normalize to BCNF

Given Relation

PATIENT(PID, PName, DocID, DocName, Specialization, WardNo, WardName)

This relation stores patient, doctor, and ward information together.

Step 1: Functional Dependencies

Assume:

- Each patient has one patient name.
- Each patient is assigned to one doctor.
- Each doctor has one name and specialization.
- Each patient is assigned to one ward.
- Each ward has one ward name.

Therefore:

1. PID → PName
2. PID → DocID
3. PID → WardNo
4. DocID → DocName
5. DocID → Specialization
6. WardNo → WardName

Candidate Key

PID uniquely identifies a patient.

Therefore:

Candidate Key = PID

Step 2: Identify Anomalies

Update Anomaly

Suppose Dr. D01 changes specialization from Cardiology to Neurology.

If the doctor treats 50 patients, the specialization has to be changed in 50 records.

Insertion Anomaly

A doctor cannot be added unless a patient is assigned to that doctor.

Deletion Anomaly

If the last patient of a doctor is deleted, information about that doctor may also disappear.

Step 3: BCNF Condition

A relation is in BCNF when:

For every non-trivial FD X → Y, X must be a superkey.

Consider:

DocID → DocName, Specialization

But DocID is **not a superkey** of the original PATIENT relation because many patients can have the same doctor.

Therefore, the relation violates BCNF.

Similarly:

WardNo → WardName

WardNo is not a superkey of PATIENT.

Step 4: Decomposition

PATIENT

Patient(PID, PName, DocID, WardNo)

Primary Key: PID

DOCTOR

Doctor(DocID, DocName, Specialization)

Primary Key: DocID

WARD

Ward(WardNo, WardName)

Primary Key: WardNo

BCNF Verification

Patient:

PID → PName, DocID, WardNo

PID is the key.

Doctor:

DocID → DocName, Specialization

DocID is the key.

Ward:

WardNo → WardName

WardNo is the key.

Therefore, all determinants are candidate keys/superkeys.

Final BCNF Schema

PATIENT(PID, PName, DocID, WardNo)

DOCTOR(DocID, DocName, Specialization)

WARD(WardNo, WardName)

Justification

The decomposition separates patient, doctor, and ward information. It eliminates redundancy and prevents update, insertion, and deletion anomalies. Each relation satisfies the BCNF condition.

3. Case study: A university stores Enrollment(SID, SName, CourseID, CourseName, Faculty, Grade). Analyze FDs and apply 2NF decomposition.

University Enrollment — Apply 2NF Decomposition

Given Relation

ENROLLMENT (SID, SName, CourseID, CourseName, Faculty, Grade)

The university stores student and course details together with enrollment grades.

Step 1: Functional Dependencies

Assume:

1. SID → SName
2. CourseID → CourseName
3. CourseID → Faculty
4. (SID, CourseID) → Grade

Candidate Key

A student can enroll in many courses, and a course can have many students.

Therefore, neither SID nor CourseID alone identifies an enrollment.

Candidate key:

(SID, CourseID)

Step 2: 1NF

The relation is in 1NF because all attributes contain atomic values.

Step 3: Identify Partial Dependencies

The composite key is:

(SID, CourseID)

But:

SID → SName

means SName depends only on SID.

Also:

CourseID → CourseName, Faculty

means CourseName and Faculty depend only on CourseID.

These are partial dependencies.

Therefore:

ENROLLMENT is not in 2NF.

Step 4: Decompose

STUDENT

Student(SID, SName)

Primary Key: SID

COURSE

Course(CourseID, CourseName, Faculty)

Primary Key: CourseID

ENROLLMENT

Enrollment(SID, CourseID, Grade)

Primary Key: (SID, CourseID)

Step 5: Verify 2NF

Student:

SID → SName

SID is the complete key.

Course:

CourseID → CourseName, Faculty

CourseID is the complete key.

Enrollment:

(SID, CourseID) → Grade

Grade depends on the complete composite key.

Therefore, there are no partial dependencies.

Example

Suppose the original table contains:

SID	SName	CourseID	CourseName	Faculty	Grade
S01	Arun	C01	DBMS	Dr. Ravi	A
S01	Arun	C02	AI	Dr. Kumar	B
S02	Priya	C01	DBMS	Dr. Ravi	A+

Student name is repeated for every course.

Course information is also repeated for every student.

After decomposition:

Student

SID	SName
S01	Arun
S02	Priya

Course

CourseID	CourseName	Faculty
C01	DBMS	Dr. Ravi
C02	AI	Dr. Kumar

Enrollment

SID	CourseID	Grade
S01	C01	A
S01	C02	B
S02	C01	A+

Final Answer

STUDENT(SID, SName)

COURSE(CourseID, CourseName, Faculty)

ENROLLMENT(SID, CourseID, Grade)

The decomposition eliminates partial dependencies and places the relation in 2NF.

4.Given the airline booking case: Booking(FlightNo, PassID, PassName, Seat, DeptCity, ArrCity, Date). Normalize up to BCNF.

Given Relation

BOOKING (FlightNo, PassID, PassName, Seat, DeptCity, ArrCity, Date)

The relation stores passenger, flight, and booking information in one table.

Step 1: Functional Dependencies

Assume:

1. PassID → PassName
2. FlightNo → DeptCity
3. FlightNo → ArrCity
4. (FlightNo, Date, PassID) → Seat

Therefore:

PassID → PassName

FlightNo → DeptCity, ArrCity

(FlightNo, Date, PassID) → Seat

Candidate Key

A passenger may book multiple flights.

A flight may have many passengers.

A flight also operates on different dates.

Therefore:

Candidate Key = (FlightNo, Date, PassID)

Step 2: 1NF

The relation is in 1NF because all attributes are atomic.

Step 3: 2NF

The composite key is:

(FlightNo, Date, PassID)

But:

PassID → PassName

This is a partial dependency.

Also:

FlightNo → DeptCity, ArrCity

These attributes depend only on part of the composite key.

Therefore, the relation is not in 2NF.

Step 4: Decompose

PASSENGER

Passenger(PassID, PassName)

Primary Key: PassID

FLIGHT

Flight(FlightNo, DeptCity, ArrCity)

Primary Key: FlightNo

BOOKING

Booking(FlightNo, Date, PassID, Seat)

Primary Key:

(FlightNo, Date, PassID)

Step 5: BCNF Verification

Passenger

PassID → PassName

PassID is the key.

Therefore, BCNF.

Flight

FlightNo → DeptCity, ArrCity

FlightNo is the key.

Therefore, BCNF.

Booking

(FlightNo, Date, PassID) → Seat

The determinant is the candidate key.

Therefore, BCNF.

Additional Constraint

A seat should normally be unique for a particular flight and date:

(FlightNo, Date, Seat) → PassID

If this business rule applies, (FlightNo, Date, Seat) becomes another candidate key for BOOKING.

Final BCNF Design

PASSENGER(

PassID,
PassName
)

FLIGHT(
FlightNo,
DeptCity,
ArrCity
)

BOOKING(
FlightNo,
Date,
PassID,
Seat
)

Anomalies Removed

Update: Flight city information is stored once.

Insertion: A flight can be added without first creating a booking.

Deletion: Deleting a booking does not delete passenger or flight details.

Conclusion

The decomposition removes partial dependencies and ensures that every determinant is a superkey. Hence, the final design satisfies BCNF.

5.Analyze an HR system schema for a multinational company. Identify all functional and transitive dependencies and normalize to 3NF.

Given/Assumed HR Relation

For a multinational company, consider:

EMPLOYEE (EmpID, EmpName, DeptID, DeptName, ManagerID, ManagerName, CountryID, CountryName, GradeID, SalaryRange)

This is a typical poorly normalized HR relation.

Step 1: Identify Functional Dependencies

Employee dependency

EmpID → EmpName

Employee ID uniquely determines employee name.

Department dependency

DeptID → DeptName

Department ID determines department name.

Manager dependency

ManagerID → ManagerName

Manager ID determines manager name.

Country dependency

CountryID → CountryName

Country ID determines country name.

Grade dependency

GradeID → SalaryRange

Grade ID determines the salary range.

Therefore, the major FDs are:

EmpID → EmpName

DeptID → DeptName

ManagerID → ManagerName

CountryID → CountryName

GradeID → SalaryRange

Also:

EmpID → DeptID

EmpID → ManagerID

EmpID → CountryID

EmpID → GradeID

Candidate Key

EmpID uniquely identifies each employee.

Therefore:

Candidate Key = EmpID

Step 2: Identify Transitive Dependencies

A transitive dependency occurs when:

A → B and B → C

therefore:

A → C

Department

EmpID → DeptID

DeptID → DeptName

Therefore:

EmpID → DeptName

This is a transitive dependency.

Manager

EmpID → ManagerID

ManagerID → ManagerName

Therefore:

EmpID → ManagerName

This is transitive.

Country

EmpID → CountryID

CountryID → CountryName

Therefore:

EmpID → CountryName

This is transitive.

Grade

EmpID → GradeID

GradeID → SalaryRange

Therefore:

EmpID → SalaryRange

This is also transitive.

Step 3: Problems in the Original Relation

Update Anomaly

Suppose a department changes its name.

If 100 employees belong to that department, the department name has to be updated in 100 records.

Insertion Anomaly

A new department cannot be stored unless an employee is assigned to it.

Deletion Anomaly

If the last employee belonging to a department is deleted, department information may also be lost.

Step 4: Normalize to 3NF

EMPLOYEE

Employee(
  EmpID,
  EmpName,
  DeptID,
  ManagerID,
  CountryID,
  GradeID
)

Primary Key: EmpID

DEPARTMENT

Department(
  DeptID,
  DeptName
)

Primary Key: DeptID

MANAGER

Manager(
  ManagerID,
  ManagerName
)

Primary Key: ManagerID

COUNTRY

Country(
  CountryID,
  CountryName
)

Primary Key: CountryID

SALARY GRADE

SalaryGrade(
  GradeID,

SalaryRange

)

Primary Key: GradeID

Step 5: 3NF Verification

Employee

EmpID → EmpName, DeptID, ManagerID, CountryID, GradeID

The determinant EmpID is the primary key.

Department

DeptID → DeptName

DeptID is the primary key.

Manager

ManagerID → ManagerName

ManagerID is the primary key.

Country

CountryID → CountryName

CountryID is the primary key.

SalaryGrade

GradeID → SalaryRange

GradeID is the primary key.

Thus, non-key attributes no longer depend transitively on another non-key attribute.

Final 3NF Schema

EMPLOYEE(

EmpID,

EmpName,

DeptID,

ManagerID,

CountryID,

GradeID

)

DEPARTMENT(

DeptID,

DeptName

)

MANAGER(

```
    ManagerID,
    ManagerName
)

COUNTRY(
    CountryID,
    CountryName
)

SALARY_GRADE(
    GradeID,
    SalaryRange
)
```

6.Case: Library (MemberID, MemberName, BookID, Title, Author, Genre, BorrowDate). Identify MVDs and decompose to 4NF.

Given Relation

LIBRARY (MemberID, MemberName, BookID, Title, Author, Genre, BorrowDate)

The library relation stores member information, book information, and borrowing information in one relation.

Step 1: Identify Functional Dependencies

Assume:

1. MemberID → MemberName
2. BookID → Title
3. BookID → Author
4. BookID → Genre
5. (MemberID, BookID) → BorrowDate

Therefore:

MemberID → MemberName

BookID → Title, Author, Genre

(MemberID, BookID) → BorrowDate

The candidate key for the borrowing information is:

(MemberID, BookID)

because one member can borrow many books and one book can be borrowed by many members.

Step 2: Identify Multivalued Dependencies

A **multivalued dependency (MVD)** occurs when one attribute determines a set of independent values of another attribute.

For example, a member can borrow multiple books:

MemberID →→ BookID

This means MemberID is associated with multiple independent BookID values.

If the database also stores independent member interests/genres, another MVD may exist:

MemberID →→ Genre

The important point is that the book collection and another independent collection should not be stored together in one relation.

Step 3: Why 4NF is Violated?

A relation is in **4NF** if, for every non-trivial MVD:

$X \rightarrow\rightarrow Y$

X must be a superkey.

Here:

MemberID $\rightarrow\rightarrow$ BookID

but MemberID alone is not a superkey of the complete relation.

Therefore, the relation violates **4NF**.

Step 4: Decompose the Relation

MEMBER

Member(MemberID, MemberName)

Primary Key: MemberID

BOOK

Book(BookID, Title, Author, Genre)

Primary Key: BookID

BORROWING

Borrowing(MemberID, BookID, BorrowDate)

Primary Key:

(MemberID, BookID)

Step 5: If Genre is an Independent MVD

If the case specifically states that genres/interests are independently associated with members, create:

MEMBER_GENRE

Member_Genre(MemberID, Genre)

Primary Key:

(MemberID, Genre)

Final 4NF Design

MEMBER(MemberID, MemberName)

BOOK(BookID, Title, Author, Genre)

BORROWING(MemberID, BookID, BorrowDate)

MEMBER_GENRE(MemberID, Genre)

Justification

The original relation contains redundant combinations of member, book, and independent multivalued information. Decomposing the relation separates these independent facts and eliminates unnecessary repetition.

Therefore, the resulting relations satisfy 4NF, assuming the stated MVDs.

7.Analyze a telecom billing schema and identify all candidate keys, primary keys, and functional dependencies before normalization

Given/Assumed Relation

Consider the following telecom billing schema:

BILLING(CustomerID, PhoneNo, CustomerName, PlanID, PlanName, CallID, CallDate, Duration, Rate, Amount)

This relation contains customer, phone, plan, and call billing information.

Step 1: Identify Functional Dependencies

Customer Dependency

A customer ID identifies one customer.

CustomerID → CustomerName

Phone Dependency

Assume every phone number belongs to one customer.

PhoneNo → CustomerID

Therefore:

PhoneNo → CustomerName

transitively.

Plan Dependency

A plan ID identifies its plan name and rate.

PlanID → PlanName, Rate

Call Dependency

Each call has a unique CallID.

CallID → PhoneNo, CallDate, Duration

If each call is associated with one plan:

CallID → PlanID

The billing amount is determined by the call details and applicable rate. If the amount is stored as a derived attribute:

(CallID, Rate) → Amount

Under the simpler assumption that the plan is fixed for a call:

CallID → Amount

Step 2: Candidate Key

If CallID uniquely identifies every call, then:

Candidate Key = CallID

because:

CallID → PhoneNo

CallID → CallDate

CallID → Duration

CallID → PlanID

and through other dependencies it can determine the remaining attributes.

Step 3: Primary Key

A suitable primary key is:

CallID

because it uniquely identifies each billing transaction.

Step 4: Alternate Candidate Key

If the business rule says that phone numbers are unique for customers:

PhoneNo can be a candidate/alternate key in the CUSTOMER/PHONE relation.

However, it should not automatically be treated as a candidate key for the complete BILLING relation because one phone number can generate many calls.

Step 5: FD Summary

Functional Dependency	**Meaning**
CustomerID → CustomerName	Customer ID identifies customer
PhoneNo → CustomerID	Phone identifies customer
PlanID → PlanName, Rate	Plan identifies plan details
CallID → PhoneNo	Call identifies phone
CallID → CallDate	Call identifies date
CallID → Duration	Call identifies duration
CallID → PlanID	Call identifies plan
CallID → Amount	Call identifies billing amount

Key Classification

Candidate Key: CallID

Primary Key: CallID

Foreign Keys: PhoneNo, PlanID, CustomerID depending on the decomposed design.

Conclusion

Before normalization, identifying candidate keys and FDs is important because they reveal partial,

transitive, and other dependencies. These dependencies can then be used to normalize the telecom billing database.

8.A retail inventory system has Product(PID, PName, SupplierID, SupplierName, Category, Price, Warehouse). Apply complete normalization

Given Relation

PRODUCT (PID, PName, SupplierID, SupplierName, Category, Price, Warehouse)

The relation stores product, supplier, category, price, and warehouse information together.

Step 1: Functional Dependencies

Assume:

PID → PName

PID → SupplierID

PID → Category

PID → Price

SupplierID → SupplierName

If a product can be stored in multiple warehouses, the product-warehouse combination represents a separate relationship.

Therefore:

(PID, Warehouse)

can be treated as the inventory relationship key.

Step 2: Candidate Key

If the same product can exist in multiple warehouses:

Candidate Key = (PID, Warehouse)

Step 3: 1NF

The relation is in 1NF if:

- Each field contains a single atomic value.
- There are no repeating groups.

So the relation satisfies 1NF.

Step 4: 2NF

The composite key is:

(PID, Warehouse)

But:

PID → PName

PID → SupplierID

PID → Category

PID → Price

These attributes depend only on PID, which is part of the composite key.

Therefore, they are **partial dependencies**.

The relation is not in 2NF.

Step 5: Decompose into 2NF

PRODUCT

Product(PID, PName, SupplierID, Category, Price)

Primary Key: PID

PRODUCT_WAREHOUSE

Product_Warehouse(PID, Warehouse)

Primary Key:

(PID, Warehouse)

Step 6: Remove Transitive Dependency for 3NF

We have:

PID → SupplierID

SupplierID → SupplierName

Therefore:

PID → SupplierName

This is a transitive dependency.

So supplier information should be separated.

SUPPLIER

Supplier(SupplierID, SupplierName)

Primary Key: SupplierID

Final Normalized Design

PRODUCT(

PID,

PName,

SupplierID,
Category,
Price
)

SUPPLIER(
SupplierID,
SupplierName
)

PRODUCT_WAREHOUSE(
PID,
Warehouse
)

Optional Warehouse Relation

If warehouse has additional attributes such as location:
WAREHOUSE(WarehouseID, WarehouseName, Location)
Then:
PRODUCT_WAREHOUSE(PID, WarehouseID)
can be used.

Anomalies Removed

Update anomaly: Supplier information is stored only once.
Insertion anomaly: A supplier can be added without first adding a product.
Deletion anomaly: Deleting a product does not delete supplier information.

Conclusion

The original relation contains partial and transitive dependencies. The final decomposition removes these dependencies and gives a **3NF design**. Under the stated assumptions, each resulting relation also satisfies BCNF.

9. Study the given poorly normalized schema for a School ERP and propose a fully normalized design up to BCNF with justification.

Given Poorly Normalized Relation

Consider the School ERP relation:

SCHOOL_ERP (StudentID, StudentName, ClassID, ClassName, TeacherID, TeacherName, SubjectID, SubjectName, Grade)

The relation stores student, class, teacher, subject, and grade information together.

Step 1: Functional Dependencies

Assume:

StudentID → StudentName

StudentID → ClassID

ClassID → ClassName

TeacherID → TeacherName

SubjectID → SubjectName

SubjectID → TeacherID

(StudentID, SubjectID) → Grade

Step 2: Candidate Key

A student can study multiple subjects.

A subject can be studied by multiple students.

Therefore:

Candidate Key = (StudentID, SubjectID)

because the combination uniquely identifies a student's grade for a subject.

Step 3: Identify Partial Dependencies

Since:

StudentID → StudentName

StudentID → ClassID

StudentName and ClassID depend only on part of the composite key.

Similarly:

SubjectID → SubjectName

SubjectID → TeacherID

SubjectName and TeacherID depend only on SubjectID.

Therefore, the relation violates 2NF.

Step 4: Identify Transitive Dependencies

We have:

StudentID → ClassID

ClassID → ClassName

Therefore:

StudentID → ClassName

This is a transitive dependency.

Similarly:

SubjectID → TeacherID

TeacherID → TeacherName

Therefore:

SubjectID → TeacherName

This is another transitive dependency.

Step 5: BCNF Decomposition

STUDENT

Student(StudentID, StudentName, ClassID)

Primary Key: StudentID

CLASS

Class(ClassID, ClassName)

Primary Key: ClassID

SUBJECT

Subject(SubjectID, SubjectName, TeacherID)

Primary Key: SubjectID

TEACHER

Teacher(TeacherID, TeacherName)

Primary Key: TeacherID

ENROLLMENT

Enrollment(StudentID, SubjectID, Grade)

Primary Key:

(StudentID, SubjectID)

Step 6: BCNF Verification

Student

StudentID → StudentName, ClassID

StudentID is the key.

Therefore, BCNF.

Class

ClassID → ClassName

ClassID is the key.

Therefore, BCNF.

Subject

SubjectID → SubjectName, TeacherID

SubjectID is the key.

Therefore, BCNF.

Teacher

TeacherID → TeacherName

TeacherID is the key.

Therefore, BCNF.

Enrollment

(StudentID, SubjectID) → Grade

The determinant is the key.

Therefore, BCNF.

Final BCNF Design

STUDENT(StudentID, StudentName, ClassID)

CLASS(ClassID, ClassName)

SUBJECT(SubjectID, SubjectName, TeacherID)

TEACHER(TeacherID, TeacherName)

ENROLLMENT(StudentID, SubjectID, Grade)

Justification

The decomposition eliminates:

- Partial dependencies
- Transitive dependencies
- Repeated student information
- Repeated teacher information
- Repeated class information
- Update anomalies
- Insertion anomalies
- Deletion anomalies

Therefore, the final School ERP database is normalized up to BCNF.

10.Given a movie streaming platform schema, identify join dependencies and decompose to 5NF with lossless-join verification.

Given/Assumed Relation

Consider:

STREAMING (MovieID, ActorID, PlatformID)

The relation records which actors are associated with movies and which platforms stream those movies.

Step 1: Identify the Relationships

There are three entities:

- Movie
- Actor
- Streaming Platform

The original relation represents a three-way association:

MovieID + ActorID + PlatformID

For example:

MovieID	ActorID	PlatformID
M1	A1	P1
M1	A1	P2
M1	A2	P1
M1	A2	P2

Step 2: Identify Join Dependency

Suppose the three relationships are independent:

1. A movie has actors.
2. A movie is available on platforms.
3. An actor is associated with platforms.

The join dependency can be represented as:

*(MovieID, ActorID),
(MovieID, PlatformID),
(ActorID, PlatformID)

This indicates that the original relation can be reconstructed by joining these smaller relations.

Step 3: Why Decomposition is Required

Suppose Movie M1 has:

- Actors A1 and A2
- Platforms P1 and P2

The original table may need to store four combinations:

M1-A1-P1

M1-A1-P2

M1-A2-P1

M1-A2-P2

This creates redundancy.

If the relationships are independent, it is better to store them separately.

Step 4: 5NF Decomposition

MOVIE_ACTOR

Movie_Actor(MovieID, ActorID)

Primary Key:

(MovieID, ActorID)

MOVIE_PLATFORM

Movie_Platform(MovieID, PlatformID)

Primary Key:

(MovieID, PlatformID)

ACTOR_PLATFORM

Actor_Platform(ActorID, PlatformID)

Primary Key:

(ActorID, PlatformID)

Step 5: Lossless Join Verification

The original relation can be reconstructed using:

Movie_Actor

⋈

Movie_Platform

⋈

Actor_Platform

If the resulting relation contains exactly the valid tuples of the original relation, the decomposition is **lossless**.

In relational algebra:

R =

Movie_Actor

⋈ Movie_Platform

⋈ Actor_Platform

Therefore, the decomposition preserves the required join relationship under the assumed join dependency.

Step 6: 5NF Condition

A relation is in **5NF (Project-Join Normal Form)** when every non-trivial join dependency is implied by candidate keys.

After decomposition, the independent relationships are stored separately:

MOVIE_ACTOR

MOVIE_PLATFORM

ACTOR_PLATFORM

This eliminates unnecessary redundancy caused by the three-way relationship.

Final 5NF Design

MOVIE_ACTOR(MovieID, ActorID)

MOVIE_PLATFORM(MovieID, PlatformID)

ACTOR_PLATFORM(ActorID, PlatformID)

Justification

The original three-way relation contains redundancy because combinations of movies, actors, and platforms may be independently generated. Decomposing the relation according to the join dependency reduces redundancy and produces a lossless decomposition under the stated assumptions.

Therefore, the schema is normalized to 5NF.